BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 110 · Bubble Sort

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What must be true before binary search is used?

Q2. What does the MAR hold?
A. The last calculation result
B. The address being accessed
C. Data moving from memory
D. All startup instructions

Q3. “Functions can return only one digit and can never return an array.” Correct this. Explain why.

Q4. A café program finds the cost of identical drinks. “Inputs are the final results.” Correct this. Give the inputs, process and output.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Bubble sort — Sorting by repeatedly comparing and swapping neighbouring values.

Pass — A sweep through adjacent comparisons.

Swap — Exchange the positions of two values.

Ascending order — Smallest to largest.

New learning

Compare neighbours: Compare each adjacent pair in turn during a pass.

Swap when out of order: For ascending order, swap when the left element is greater than the right.

A complete pass: After one full left-to-right pass, the largest element in the unsorted part has bubbled right.

Repeat until sorted: Subsequent passes continue until no swaps are needed or the passes are complete.

Worked example

Show the array after each comparison in the FIRST pass of bubble sort on [5,2,8,1,4]. [5 marks]

1. Compare 5 and 2

2. Swap to [2,5,8,1,4].

3. Compare 5 and 8: no swap. Then compare 8 and 1: swap → [2,5,1,8,4].

4. Compare 8 and 4: swap → [2,5,1,4,8]. Largest value 8 is now at right. [5]

We Do · Practise with guidance

Guided 1

Show the array after ONE pass of bubble sort on [4,1,3,2]. [4]

Worked method: [1,4,3,2] [1]; [1,3,4,2] [1]; [1,3,2,4] [1]; 4 at right end [1].

Guided 2

Explain why [1,2,3,4] needs no swaps in an ascending bubble-sort pass. [3]

Worked method: Every left-hand element is already ≤ its neighbour [1]; no pair is out of order [1]; zero swaps in the pass [1].

Guided 3

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

An array is [3,1,2]. Apply a complete pass and explain the result. [4]

Worked method: 3>1 swap → [1,3,2] [2]; 3>2 swap → [1,2,3] [1]; largest element 3 at end [1].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Code First · Predict → Write → Test → Improve

Correct the ascending-order comparison before a bubble-sort swap.

```
array sizes = [6,2]
if sizes[0] < sizes[1] then
    temp = sizes[0]
    sizes[0] = sizes[1]
    sizes[1] = temp
endif
print(sizes[0])
```

iterate

Question 1

Give the array after one complete left-to-right bubble-sort pass on [7,2,5,1]. [4] Source: Original OCR J277-style practice question.

Question 2

Explain why bubble sort compares adjacent items. [3] Source: Original OCR J277-style practice question.

Question 3

State one way to detect that a bubble sort can finish early. [2] Source: Original OCR J277-style practice question.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

Binary search can safely discard half only when data is sorted.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. The list must be sorted. This lets the search safely rule out half the remaining items.

Q2. B — The address being accessed

Q3. Yes. A subprogram can receive an array; a function can return an array.

Q4. Inputs: price and quantity. Processing: multiply them. Output: total cost. Inputs come before the calculation; the output comes after it.

I Do / We Do pages · Guided practice

Q1. [1,4,3,2] [1]; [1,3,4,2] [1]; [1,3,2,4] [1]; 4 at right end [1].

Q2. Every left-hand element is already ≤ its neighbour [1]; no pair is out of order [1]; zero swaps in the pass [1].

Q3. 3>1 swap → [1,3,2] [2]; 3>2 swap → [1,2,3] [1]; largest element 3 at end [1].

You Do pages · Code First

```
array sizes = [6,2]
if sizes[0] > sizes[1] then
    temp = sizes[0]
    sizes[0] = sizes[1]
    sizes[1] = temp
endif
print(sizes[0])
```

Expected result: 2

You Do pages · Independent questions

Q1. 7/2 swap [2,7,5,1] [1]; 7/5 swap [2,5,7,1] [1]; 7/1 swap [2,5,1,7] [1]; final [2,5,1,7] [1].

Q2. It checks whether each neighbouring pair is in order [1]; swaps an out-of-order pair [1]; repeated passes gradually put values into sorted positions [1].

Q3. Complete a pass with zero swaps [1]; array is already sorted [1].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Model answer

True

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: